Slide 1, title (20 seconds), **Kapeesh** Hi everyone, we're Team 32 from Plymouth University, I'm Kapeesh and I'm here with Jorjit and Hussain, and our client for this project was Yvonne Kam. What we've built is a clinical decision support system for cardiovascular risk, but instead of using a static calculator like most tools out there, we've built it on top of a Bayesian Network.

Slide 2, the problem (35 seconds), **Kapeesh** So the issue with standard CVD calculators is that they give you a flat percentage and don't show how symptoms actually interact with each other, which is a problem because clinicians need to see how things like blood pressure, cholesterol, age and ECG propagate through a network. On top of that, the UCI dataset we worked with only has 303 records, so unusual combinations end up producing paradoxical risk scores, and if you try to plug in a standard LLM to explain things, you get hallucination risk in a medical setting. Our solution pairs the Bayesian Network with a dual-AI safety architecture to handle both problems at once.

Slide 3, delivery (25 seconds), **Hussain** We delivered the whole project in six sprints across 14 weeks. Semester one was about laying the data foundation and getting a Lovable AI prototype out the door, and semester two was where we rebuilt everything in React and FastAPI, containerised it with Docker, and integrated Gemini llm. Three of us on the team, weekly sprints, 13 client meetings, and we did all of this on a zero budget using open-source tools.

Slide 4, the model (35 seconds), **Jorjit** This slide covers our biggest technical pivot of the project. We started with a multi-tiered network, but it caused probability dilution and capped our risk scores at around 35%, which obviously wasn't going to work clinically. So we flattened the topology into a Naive Bayes star, where the disease target connects directly to 13 clinical nodes, and the posterior risk is the proportional product of those symptom probabilities. We also added a BDeu prior with an effective sample size of one, which stops the network from crashing on zero-probability cases. After all that, the final model came out at 87% accuracy with a 1.75% error rate.

Slide 5, architecture (25 seconds), **Hussain** On the architecture side, we've got a React frontend running on port 3000 and a FastAPI backend on port 8000, with four endpoints, predict, ask-ai, ask-ai-general, and chat-ai. The model is persisted as a pickled file so the server boots almost instantly, and we're using postgreSQL for patient records dynamically which is connected to our hosted server. Everything is containerised with Docker Compose and hosted on Render, with a small ping bot running to keep the server awake.

Slide 6, prototypes (25 seconds), **Kapeesh** We went through three prototype iterations to get here. The first one was a CSV-driven 14-node graph, which worked but was way too technical for actual clinical use. The second was a Lovable AI MVP, where we introduced semantic labels and the Top Influential Factors panel. The third and final version is the React and FastAPI build, with treatment intervention nodes and a properly mobile-responsive layout.

Slide 7, dual-AI safety (40 seconds), **Kapeesh** This is the safety layer, and it's where most of the design thinking went. Path one is the Aligned AI Explainer, which is strictly grounded in the Bayesian Network's math, so it can't go off and invent its own numbers. Path two is an

independent Gemini opinion that evaluates the patient blindly, with no access to the classifier at all, so you get a genuinely independent second view. Path three is an agentic chat doctor with function-calling, and we hard-coded its scope down to three predefined interventions. If path one and path two disagree, the system flags the contradiction to the clinician rather than hiding it.

Slide 8, the delivered system (30 seconds), **Hussain** The delivered system starts with a login authentication gateway, then an interactive risk calculator with 13 patient attributes and three treatment interventions. The results come back in a three-panel view, the Bayesian score, the Aligned Explainer, and the Independent Opinion. We've also got a risk factor breakdown using leave-one-out impact, a consensus scatter plot comparing the two AI views, and a verification panel showing 84.5% accuracy, 81% sensitivity and 87.5% specificity.

Slide 9, testing (25 seconds), **Jorjit** On testing, we ran glass-box validation across every probability distribution in the network, and the CPTs were checked against the sum-to-one rule. The backend was tested with pytest and the frontend was tested for state management. We also ran a three-stage user acceptance test with medical students, and the final metrics landed at 87% accuracy and an MSE of 0.0175.

Slide 10, LSEP compliance (25 seconds), **Kapeesh** For LSEP compliance, on the legal side we've got a GDPR gateway, Docker security, and a clean IP handover. Socially, we addressed dataset bias and replaced the cryptic codes with natural language labels. Ethically, we anonymised patient data, prioritised Bayesian transparency over black-box ML, and built in dual-AI contradiction flagging. Professionally, we ran agile, kept strict Git tracking, did 13 client meetings, and delivered a full technical handover tutorial.

Slide 11, Kapeesh reflection (20 seconds), **Kapeesh** On a personal note, I built a multi-tool AI orchestration workflow across Manus, Claude, Gemini and Fathom to help fast track all the research and design documents, and I coordinated all 13 of our client meetings. The hardest call I had to make was discarding the Lovable prototype that I engineered and rebuilding from scratch in React and FastAPI, which was uncomfortable at the time but architecturally the right move in the long run.

Slide 12, Jorjit reflection (20 seconds), **Jorjit** For me, I built the mathematical core in Python using pgmpy, and then transitioned the model to the flattened Naive Bayes classifier when we hit probability dilution. I also solved the zero-probability crash using BDeu priors, engineered the FastAPI layer, and wrote the strict prompt grounding logic for the Aligned AI Explainer. I also set up the server function so the client could access it from wherever by connecting the backend api to the fronend on render.

Slide 13, Hussain reflection (20 seconds), **Hussain** On my side,I built the initial prototype and the current React frontend from scratch and managed state across the forms and AI panels.I also fixed the Gemini API timeout issues by adding controlled delays between calls. I also wrote most of the technical documentation, and honestly, my biggest takeaway is validating the legality issues in our final project which led to major changes in the project.

Close (15 seconds), **Kapeesh** So that's our system, a transparent, mathematically grounded, dual-AI safety architecture for cardiovascular risk. Thanks for listening,